

221900180 田永铭模式识别第三次作业

1. 图像滤波操作练习

(a) (1) 核心代码：上面是转化为灰度代码，下面分别是均值滤波和高斯滤波实现。

```
img = Image.open('../图1.png') # 读取图像文件
img_array = np.array(img) # 将图像转换为numpy数组，方便进行数学运算
# 将图像转化为灰度图像，使用标准的亮度转换公式  $Y = 0.299R + 0.587G + 0.114B$ 
gray_img_array = np.dot(img_array[..., :3], [0.299, 0.587, 0.114])

def mean_filter(image, window_size): # 均值滤波
    # 创建一个大小为 window_size x window_size 的均值滤波器核
    kernel = np.ones((window_size, window_size)) / (window_size * window_size)
    # 使用卷积操作应用滤波器
    filtered_image = convolve2d(image, kernel, mode='same', boundary='symm')
    return filtered_image

def gaussian_kernel(window_size, sigma): # 高斯核
    ax = np.arange(-window_size // 2 + 1., window_size // 2 + 1.) # 创建一个坐标网格
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2. * sigma**2)) # 计算高斯函数
    return kernel / np.sum(kernel) # 归一化，确保核的总和为1

def gaussian_filter(image, window_size, sigma): # 高斯滤波
    kernel = gaussian_kernel(window_size, sigma) # 生成高斯核
    filtered_image = convolve2d(image, kernel, mode='same', boundary='symm') # 应用卷积
    return filtered_image
```

(2) 实验结果：



(3) 观察和总结：平滑效果与窗口大小的关系：两种滤波方法的平滑（模糊）效果都随着窗口尺寸的增大而增强。窗口越大，参与计算的邻域像素越多，图像细节丢失越严重，图像也就越模糊；均值滤波 vs. 高斯滤波：均值滤波器捕捉细节能力不如高斯滤波器，高斯滤波器更加平滑，可以看到 3*3 的高斯滤波器比 3*3 的均值滤波器展示的“头发丝”的细节更好；边缘保留：对于较大的窗口（如 15x15），均值滤波对图像边缘的破坏比高斯滤波更明显。

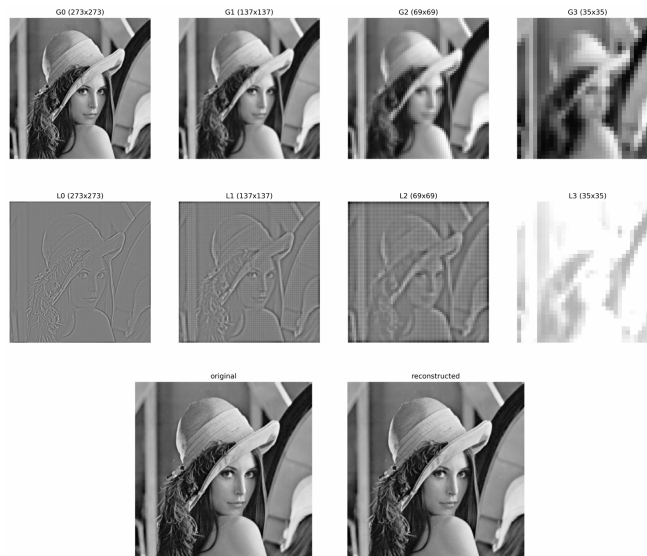
(b) (1) 核心代码：上面是构建金字塔代码，下面是重建代码。

```
num_levels = 4 # --- 构建高斯金字塔 --- 层数
gaussian_pyramid = [gray_img_array.copy()]
current_image = gray_img_array.copy()
for i in range(num_levels - 1): # 循环生成金字塔的每一层
    blurred_image = gaussian_filter(current_image, 5, 1.0) # 这里我们使用一个5x5的核, sigma=1,
    downsampled_image = downsample(blurred_image) # 降采样
    gaussian_pyramid.append(downsampled_image) # 将结果存入金字塔列表
    current_image = downsampled_image

laplacian_pyramid = [] # --- 构建拉普拉斯金字塔 ---
for i in range(num_levels - 1): # 循环生成拉普拉斯金字塔的每一层
    G_i = gaussian_pyramid[i]
    G_i_plus_1 = gaussian_pyramid[i+1] # 获取当前层 G_i 和下一层 G_i+1
    upsampled_G = upsample(G_i_plus_1, G_i.shape) # 将 G_i+1 上采样到 G_i 的尺寸
    blurred_upsampled_G = gaussian_filter(upsampled_G, 5, 1.0) * 4 # 用卷积核平滑上采样后的图像
    L_i = G_i - blurred_upsampled_G # 计算差值, 得到拉普拉斯层
    laplacian_pyramid.append(L_i)

reconstructed_image = laplacian_pyramid[-1] # --- 从金字塔恢复原图 --- # 从最顶层开始
for i in range(num_levels - 2, -1, -1): # 从上到下逐层恢复
    upsampled_rec = upsample(reconstructed_image, laplacian_pyramid[i].shape) # 1. 对当前恢复的图像进行上采样
    blurred_upsampled_rec = gaussian_filter(upsampled_rec, 5, 1.0) * 4 # 2. 用高斯核模糊
    reconstructed_image = blurred_upsampled_rec + laplacian_pyramid[i] # 3. 加上同层的拉普拉斯图像
```

(2) 实验结果：三排依次是：高斯金字塔，拉普拉斯金字塔，原图像 vs 重建图像。



(c) (1) 核心代码：主要利用了 FFT 和 IFFT 变换，本页这里代码是主函数流程，下一页中代码是基础实现部分。这里主要展示均值滤波，高斯滤波因为高斯函数频域还是高斯函数，所以很方便，与 1.(a) 中类似。

```
def show_results(img, H):
    F = dft2(img) # 首先应用傅里叶变换
    filtered_freq = F * H # 在频域中应用滤波器
    filtered_img = idft2(filtered_freq) # 转换回空间域
```

```

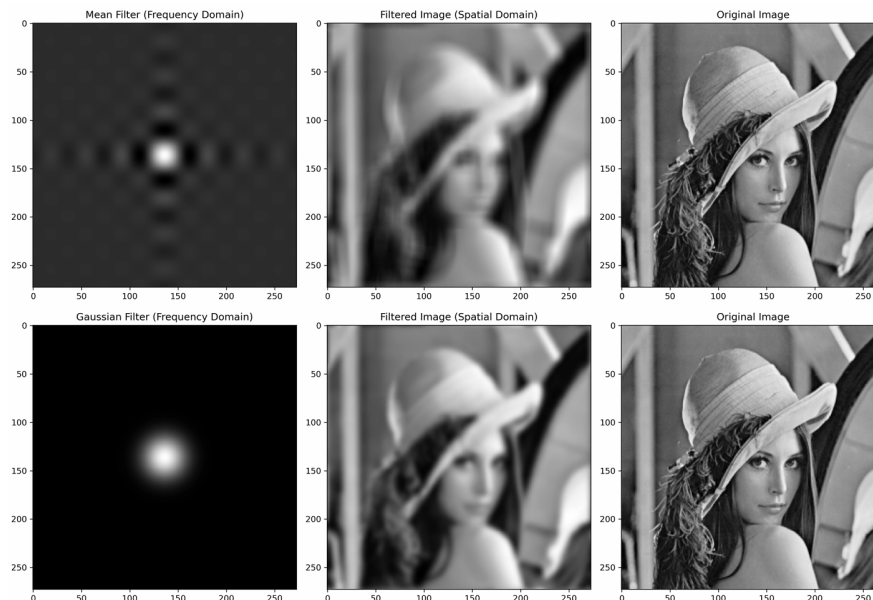
def dft2(img):
    M, N = img.shape # 二维离散傅里叶变换, 中心化
    x = np.arange(M) # 生成频率索引
    y = np.arange(N) # 生成频率索引
    u = x.reshape((M, 1)) # 将 x 转换为列向量
    v = y.reshape((N, 1)) # 将 y 转换为列向量
    W_M = np.exp(-2j * np.pi * u @ x[np.newaxis, :] / M) # MxM # DFT 矩阵
    W_N = np.exp(-2j * np.pi * v @ y[np.newaxis, :] / N) # NxN
    F = W_M @ img @ W_N.T # 先按行DFT, 再按列DFT
    shift = np.fromfunction(lambda i, j: (-1)**(i + j), img.shape) # 中心化: 乘以 (-1)^(x+y)
    return F * shift

def idft2(freq_img): # 二维离散傅里叶逆变换, 中心化
    M, N = freq_img.shape # 获取频域图像的形状
    u = np.arange(M) # 生成频率索引
    v = np.arange(N) # 生成频率索引
    x = u.reshape((M, 1)) # 将 u 转换为列向量
    y = v.reshape((N, 1)) # 将 v 转换为列向量
    W_M_inv = np.exp(2j * np.pi * x @ u[np.newaxis, :] / M) # MxM # IDFT 矩阵
    W_N_inv = np.exp(2j * np.pi * y @ v[np.newaxis, :] / N) # NxN
    shift = np.fromfunction(lambda i, j: (-1)**(i + j), freq_img.shape) # 去中心化: 乘以 (-1)^(x+y)
    freq_img_shifted = freq_img * shift
    img_recon = W_M_inv @ freq_img_shifted @ W_N_inv.T # 先按行IDFT, 再按列IDFT
    img_recon /= (M * N)
    return np.abs(img_recon)

def make_mean_filter(shape, kernel_size): # 均值滤波器
    rows, cols = shape # 获取图像的行数和列数
    u = np.fft.fftfreq(rows) # np.fft.fftfreq 会生成一个适合傅里叶变换的频率序列
    v = np.fft.fftfreq(cols)
    V, U = np.meshgrid(v, u)
    sinc_u = np.sinc(U * kernel_size) # 空间域的矩形函数, 其傅里叶变换是 sinc 函数。
    sinc_v = np.sinc(V * kernel_size)
    H = sinc_u * sinc_v # 2D sinc 函数是两个1D sinc函数的乘积
    return np.fft.fftshift(H) # fftfreq 生成的频率已经是“未移位”的顺序, 所以直接计算得到的 H

```

(2) 实验结果：两排分别是均值滤波器以及高斯滤波器下的：频域滤波结果、空域结果、原始图像。



(3) 观察和总结：均值滤波：左图是星芒/十字状的 sinc 函数，中间是一个方框，上下左右有明暗相间条纹，中图是被模糊且可能带有一点振铃效应的图像；高斯滤波：左图是中心亮、边缘平滑过渡的光斑，较远的周围不再有亮点，中图是被自然、平滑地模糊了的图像。

2. 图像角点检测

(a) (1) 核心代码：下图是核心代码。主要按照红色线分为三个部分。分别是：计算基础的梯度、矩阵等等基础信息；非极大值抑制；寻找潜在角点；为下一个尺度做准备。

```
for scale_level in range(num_scales): # 遍历每个尺度
    ix = cv2.Sobel(current_image, cv2.CV_64F, 1, 0, ksize=3) # 计算x方向梯度
    iy = cv2.Sobel(current_image, cv2.CV_64F, 0, 1, ksize=3) # 计算y方向梯度
    ixx = ix * ix; iyy = iy * iy; ixy = ix * iy # 计算梯度的二阶矩
    A = cv2.GaussianBlur(ixx, (window_size, window_size), 1.5) # 对二阶矩进行高斯模糊
    B = cv2.GaussianBlur(iyy, (window_size, window_size), 1.5)
    C = cv2.GaussianBlur(ixy, (window_size, window_size), 1.5)
    det_M = (A * B) - (C**2); trace_M = A + B # 计算Harris矩阵的行列式和迹
    harris_response = det_M - k * (trace_M**2) # 计算Harris响应函数
    threshold = score_threshold * harris_response.max() # 非极大值抑制
    potential_corners = harris_response > threshold
    coords = np.array(potential_corners.nonzero()).T
    values = harris_response[potential_corners]
    sorted_indices = np.argsort(values)[::-1]
    coords = coords[sorted_indices] # 按响应值降序排列，只保留最大的一些角点
    current_scale_ratio = (scale_factor ** scale_level) # 当前尺度的缩放比例
    scale_corners = []
    suppressed = np.zeros(harris_response.shape, dtype=bool)
    for r, c in coords: # 遍历所有潜在角点
        if suppressed[r, c]: # 如果该点已被抑制，则跳过
            continue
        suppression_radius = int(window_size / 2 * current_scale_ratio) # 计算抑制半径
        original_x = int(c * current_scale_ratio) # 计算原始图像中的x坐标
        original_y = int(r * current_scale_ratio) # 计算原始图像中的y坐标
        scale_radius = int(2 * current_scale_ratio) # 计算尺度半径
        scale_corners.append((original_x, original_y, scale_radius)) # 保存角点及其尺度倍
        r_min = max(0, r - suppression_radius)
        r_max = min(harris_response.shape[0], r + suppression_radius + 1)
        c_min = max(0, c - suppression_radius)
        c_max = min(harris_response.shape[1], c + suppression_radius + 1) # 计算抑制区域
        suppressed[r_min:r_max, c_min:c_max] = True # 抑制该区域内的
    all_corners.extend(scale_corners) # 保存所有尺度的角点
    if scale_level < num_scales - 1: # 为下一个尺度准备图像
        blurred_image = cv2.GaussianBlur(current_image, (window_size, window_size), 1.5)
        new_height = int(current_image.shape[0] / scale_factor)
        new_width = int(current_image.shape[1] / scale_factor)
        current_image = cv2.resize(blurred_image, (new_width, new_height), interpolation=cv2.INTER_AREA)
```

(2) 实验结果：下图是实验结果，成功检测了图中各个地方的角点。



图 1: 角点检测实验结果

3. 相机模型 (a) (1) 核心代码：我选取的参数是 $tx=40$ (x-平移)、 $ty=-30$ (y-平移)、 $angle=-25$ (顺时针旋转)、 $scale_x=0.9$ (x-缩放)、 $scale_y=1.1$ (y-缩放)、 $shear_x=0.2$ (x-错切)。我借用了双线性插值平滑图像。

```
def perform_affine_transform(img, tx, ty, angle, scale_x, scale_y, shear_x):
    h, w = img.shape[:2] # 获取图像的高度和宽度
    center = (w // 2, h // 2) # 图像中心坐标
    rad = np.deg2rad(angle)
    cos_a = np.cos(rad); sin_a = np.sin(rad) # 将角度转换为弧度
    M_rot = np.array([[cos_a, -sin_a], [sin_a, cos_a]]) # 旋转矩阵
    M_scale = np.array([[scale_x, 0], [0, scale_y]]) # 缩放矩阵
    M_shear = np.array([[1, shear_x], [0, 1]]) # 错切矩阵
    M_rss = M_rot @ M_scale @ M_shear # 组合旋转、缩放和错切矩阵
    M = np.zeros((2, 3)) # 初始化仿射变换矩阵
    M[:2, :2] = M_rss # 设置仿射变换矩阵的前两行
    M[:2, 2] = (np.array([w, h]) - M_rss @ np.array([w, h])) / 2 # 平移矩阵
    M[:2, 2] += np.array([tx, ty]) # 添加平移参数
    transformed_img = np.zeros_like(img) # 初始化变换后的图像
    for y in range(h): # 遍历图像的每一行
        for x in range(w): # 遍历图像的每一列
            source_coords = np.array([x, y, 1]) # 源图像坐标
            transformed_coords = M @ source_coords # 应用仿射变换矩阵
            new_x, new_y = transformed_coords[0], transformed_coords[1] # 获取变换后的坐标
            if 0 <= new_x < w and 0 <= new_y < h: # 检查新坐标是否在图像范围内
                transformed_img[y, x] = bilinear_interpolate(img, new_x, new_y)
    return transformed_img
```

(2) 实验结果：如图 2 的左图所示。

(b) (1) 核心代码：我选取的八个参数是 $h_{11}=1.2$ (自由度 1)、 $h_{12}=-0.2$ (自由度 2)、 $h_{13}=-30$ (自由度 3)、 $h_{21}=0.1$ (自由度 4)、 $h_{22}=1.1$ (自由度 5)、 $h_{23}=-20$ (自由度 6)、 $h_{31}=0.001$ (自由度 7, 关键参数)、 $h_{32}=0.0005$ (自由度 8, 关键参数)。我借用了双线性插值平滑图像。

```
def perform_projective_transform(img, h_params): # 执行射影变换
    h, w = img.shape[:2] # 图像的高度和宽度
    H = np.array([ # 构建单应性矩阵 H (3x3)
        [h_params['h11'], h_params['h12'], h_params['h13']],
        [h_params['h21'], h_params['h22'], h_params['h23']],
        [h_params['h31'], h_params['h32'], 1.0]
    ], dtype=float)
    transformed_img = np.zeros_like(img) # 创建一个新的空图像
    for y in range(h):
        for x in range(w):
            target_coords = np.array([x, y, 1]) # 目标坐标 (x, y) 转换为齐次坐标 (x, y, 1)
            source_coords = np.linalg.inv(H) @ target_coords # 计算源坐标
            source_x, source_y, _ = source_coords / source_coords[2] # 从齐次坐标转回实际坐标
            if 0 <= source_x < w and 0 <= source_y < h: # 确保新坐标在图像范围内
                transformed_img[y, x] = bilinear_interpolate(img, source_x, source_y)
    return transformed_img
```

(2) 实验结果：如图 2 的右图所示。

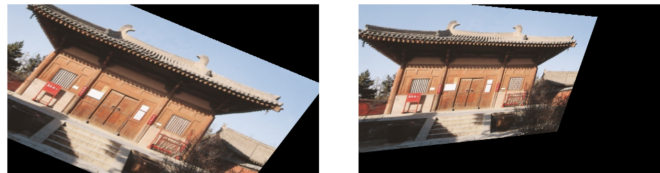


图 2: 左: 6DoF 仿射变换结果图; 右: 8DoF 射影变换结果图

(c) 伪代码如下：定义输入 P_w 是三维坐标点， K 是 3×3 的相机内参矩阵， R 是 3×3 的相机外参旋转矩阵， t 是 3×1 的相机外参平移向量。

```
FUNCTION map_world_to_image(P_w, K, R, t):
    # 1. 将世界坐标点P_w转换为齐次坐标 (4x1)
    P_w_homo = CREATE_VECTOR(P_w[0], P_w[1], P_w[2], 1)
    # 2. 构建外参矩阵 [R|t] (3x4)
    ExtrinsicMatrix = CONCATENATE_HORIZONTALLY(R, t)
    # 3. 通过外参矩阵，将世界坐标变换到相机坐标 (结果是3x1向量)
    P_cam_homo = MATRIX_MULTIPLY(ExtrinsicMatrix, P_w_homo)
    # 4. 通过内参矩阵K，将相机坐标投影到图像平面 (结果是3x1齐次图像坐标)
    p_img_homo = MATRIX_MULTIPLY(K, P_cam_homo)
    # 5. 齐次坐标归一化，得到最终的2D像素坐标 (u, v)
    w_prime = p_img_homo[2]
    IF w_prime is close to 0:
        # 点在相机光心或无穷远处，无法投影
        RETURN error "Point cannot be projected."
    END IF
    u = p_img_homo[0] / w_prime
    v = p_img_homo[1] / w_prime
    # 6. 返回最终的2D像素坐标
    p_pixel = CREATE_VECTOR(u, v)
    RETURN p_pixel
END FUNCTION
```

(d) 伪代码如下：在这里，我们仍然基于 (c) 中定义的相机内外参符号。一以下分别是透视投影、弱透视投影、正交投影的伪代码。

```
FUNCTION perspective_projection(P_w, K, R, t):
    # 将世界坐标点转换为齐次坐标 (4x1)
    P_w_homo = CREATE_VECTOR(P_w[0], P_w[1], P_w[2], 1)
    # 构建外参矩阵 [R|t] (3x4)
    ExtrinsicMatrix = CONCATENATE_HORIZONTALLY(R, t)
    # 变换并投影，得到齐次图像坐标
    p_img_homo = MATRIX_MULTIPLY(K, MATRIX_MULTIPLY(ExtrinsicMatrix, P_w_homo))
    # 归一化，得到像素坐标
    w_prime = p_img_homo[2]
    IF w_prime is close to 0:
        RETURN error "点在无穷远处或光心，无法投影"
    END IF
    u = p_img_homo[0] / w_prime
    v = p_img_homo[1] / w_prime
    RETURN CREATE_VECTOR(u, v)
END FUNCTION
```

```
FUNCTION weak_perspective_projection(P_w, K, R, t, Z_avg):
    # Z_avg: 物体在相机坐标系下的平均深度。
    # 1. 变换到相机坐标
    P_w_vec = CREATE_VECTOR(P_w[0], P_w[1], P_w[2])
    P_cam = MATRIX_MULTIPLY(R, P_w_vec) + t
    Xc, Yc, Zc = P_cam[0], P_cam[1], P_cam[2] # Zc后续将被忽略
    # 2. 使用平均深度 Z_avg 近似计算
    fx, fy = K[0, 0], K[1, 1]
    cx, cy = K[0, 2], K[1, 2]
    u = fx * (Xc / Z_avg) + cx
    v = fy * (Yc / Z_avg) + cy
    RETURN CREATE_VECTOR(u, v)
END FUNCTION
```

```
FUNCTION orthographic_projection(P_w, R, t, sx, sy, cx, cy):
    # sx, sy: 缩放因子, cx, cy: 主点偏移
    # 1. 变换到相机坐标
    P_w_vec = CREATE_VECTOR(P_w[0], P_w[1], P_w[2])
    P_cam = MATRIX_MULTIPLY(R, P_w_vec) + t
    Xc, Yc = P_cam[0], P_cam[1] # Zc被忽略
    # 2. 直接缩放和平移
    u = sx * Xc + cx
    v = sy * Yc + cy
    RETURN CREATE_VECTOR(u, v)
END FUNCTION
```